

Integration of the FINS Protocol into the Open-Source HMI System FUXA for Omron PLCs

DENNY CHRISNANDA
Universitas Bina Nusantara
jane@smith.com

FRANS SEBASTIAN*
Universitas Bina Nusantara
jane@smith.com

GILANG HANSAGITA†
Universitas Bina Nusantara
jane@smith.com

Abstract

This research aims to integrate the Factory Interface Network Service (FINS) protocol into the open-source Human–Machine Interface (HMI) system FUXA to enable direct communication with Omron Programmable Logic Controllers (PLCs). The implementation involved developing a FINS connector module in Node.js for the backend and adding a configuration interface in Angular for the frontend. Experimental results show that the system can perform read and write operations over the FINS protocol with a success rate of 99% and an average latency of 48 ms. Therefore, the enhanced FUXA system provides a stable and cost-efficient open-source HMI/SCADA solution compatible with Omron PLCs, supporting industrial automation and digital transformation initiatives.

Keywords—FINS, FUXA, HMI, Omron, PLC, Industrial Automation.

1. INTRODUCTION

Programmable Logic Controllers (PLCs) are the backbone of industrial automation systems, functioning as the primary control units that manage machinery, sensors, and actuators on the factory floor. Their reliability and deterministic behavior make them indispensable for industrial operations requiring real-time control. Among various brands, Omron PLCs are widely adopted across manufacturing sectors due to their robustness, scalability, and extensive communication options. At PT XYZ, a multinational automotive components manufacturer, over 90% of the production line equipment relies on Omron PLCs for machine control and process automation.

To enable operators to interact effectively with these automated systems, industries use *Human–Machine Interface (HMI)* systems. HMIs visualize real-time process data, provide control mechanisms, and facilitate supervisory decision-making. Traditional commercial HMI/SCADA systems, such as Wonderware or CX-Supervisor, offer powerful features for monitoring and control but often come with high licensing costs, limited flexibility, and a strong dependence on proprietary vendors. Such dependency introduces challenges like vendor lock-in, high total cost of ownership (TCO), and restricted adaptability when integrating new industrial protocols or expanding

production lines [5, 2].

In contrast, open-source HMI/SCADA platforms such as FUXA have emerged as cost-effective and flexible alternatives. FUXA is a web-based, cross-platform HMI system built on Node.js and Angular that supports modern industrial integration scenarios. Its lightweight architecture allows deployment on local servers, embedded devices, or industrial PCs, providing broad accessibility and easy customization. Open-source frameworks like FUXA are particularly attractive for small and medium-sized industries that need scalable yet affordable automation visualization tools [1].

However, a major limitation of the FUXA platform is the absence of native support for Omron’s proprietary communication protocol, the *Factory Interface Network Service (FINS)*. FINS is the standard communication protocol for Omron PLCs, enabling direct data exchange between devices and supervisory systems over UDP or TCP networks. Without FINS integration, FUXA cannot communicate with Omron PLCs, which limits its adoption in Omron-based industrial environments. This gap prevents many factories, including PT XYZ, from migrating toward open-source HMI systems despite their evident advantages in cost and flexibility.

To address this limitation, this research focuses on developing and integrating native support for the FINS protocol into the FUXA open-source HMI. The integration involves modifying both the backend and frontend layers of FUXA: implementing a new Node.js connector module (*DeviceFins*) for handling FINS

*Corresponding author

†Corresponding author

communication and extending the Angular-based user interface to configure FINS parameters such as SA1, DA1, and protocol type (UDP/TCP). By doing so, the study aims to bridge the interoperability gap between open-source HMI systems and Omron PLCs.

Therefore, the main objectives of this research are:

1. To implement FINS protocol support in the FUXA system with functionality comparable to other supported protocols such as Modbus.
2. To improve the stability and reliability of read/write communication through the FINS protocol and ensure accurate real-time data visualization on the HMI user interface.

This work contributes both practically and academically. Practically, it provides industries with a vendor-independent, open-source HMI solution that supports Omron PLCs without additional middleware or licensing fees. Academically, it demonstrates a structured approach for extending open-source industrial software through modular integration and performance validation, paving the way for future interoperability research in industrial automation.

2. RELATED WORKS

2.1. Programmable Logic Controllers (PLC) and Industrial Communication

Programmable logic controllers (PLCs) play a central role in modern industrial automation as real-time embedded systems that manage sensors, actuators, and process control logic [3]. They are designed to operate in harsh industrial environments and ensure deterministic control of machinery. To achieve interoperability among heterogeneous industrial devices, PLCs rely on various communication protocols, such as Modbus, PROFIBUS, EtherNet/IP, and Omron's Factory Interface Network Service (FINS). Each protocol has unique data addressing mechanisms, transmission layers, and reliability characteristics that determine its suitability for specific use cases. In the context of Omron PLCs—widely adopted in manufacturing—FINS enables high-speed communication and direct access to PLC memory areas such as DM, CIO, and WR, providing an efficient foundation for real-time HMI updates.

2.2. Human–Machine Interface (HMI) Systems

Human–machine interfaces (HMIs) are critical components that bridge human operators and industrial

machines. Their purpose is to visualize process variables, enable manual control, and provide diagnostics or alarm notifications [4]. Traditional HMI and SCADA systems have evolved from hardware panels to advanced software-based solutions capable of integrating data from multiple PLCs and distributed systems. Commercial HMI software—such as Omron CX-Supervisor, Siemens WinCC, or Schneider Vijeo Designer—offers robust features and stability, but at a high licensing cost. Such proprietary systems often impose vendor-specific architectures, limiting integration flexibility and leading to a phenomenon known as *vendor lock-in* [2]. Consequently, industries with limited budgets or those pursuing digital transformation initiatives increasingly explore open-source alternatives that provide both flexibility and cost efficiency.

2.3. Open-Source HMI and SCADA Platforms

The open-source movement in industrial automation has accelerated due to the need for adaptable and transparent solutions. Open-source SCADA and HMI platforms allow developers and organizations to inspect, modify, and extend system functionalities without licensing restrictions. Examples of such systems include OpenPLC, ScadaBR, and Node-RED dashboards, which have proven effective for educational, research, and small-scale industrial purposes. Among these, FUXA stands out as a modern, browser-based HMI/SCADA system developed using Node.js and Angular. It supports cross-platform deployment, real-time visualization, alarm management, and tag-based data acquisition via standard protocols such as Modbus TCP, BACnet, and MQTT [1]. Its lightweight web architecture makes FUXA suitable for cloud or IoT integration while maintaining ease of customization.

2.4. Limitations in FINS Integration with Open-Source HMI Systems

Despite its advantages, FUXA does not natively support the FINS protocol. This limitation poses a challenge in environments that rely heavily on Omron PLCs, such as PT XYZ, where more than 90% of production lines depend on Omron controllers. Previous studies and industrial reports indicate that while open-source HMI platforms commonly integrate with Modbus or OPC-UA, implementations involving FINS are scarce. Existing FINS support is mostly confined to proprietary Omron software environments or third-party libraries that are not optimized for real-time

visualization. This lack of integration hinders the adoption of open-source HMI systems in industries dominated by Omron automation infrastructure.

2.5. Research Gap and Contribution

From the literature and industrial context above, several key findings emerge:

1. Open-source HMI systems like FUXA offer flexibility and transparency but lack compatibility with certain proprietary PLC protocols.
2. FINS remains an essential communication standard in Omron-based automation but is underrepresented in open-source HMI research.
3. Previous integration studies have not focused on ensuring stable read/write communication or visual synchronization through the FINS protocol.

Therefore, this research aims to fill that gap by developing and testing a FINS connector for FUXA. The connector is designed to enable stable, bidirectional communication between FUXA and Omron PLCs via UDP/TCP while ensuring accurate data visualization in the user interface. This study extends the capabilities of open-source industrial automation systems, contributing to cost-effective and vendor-independent solutions for real-world manufacturing environments.

3. METHODOLOGY

3.1. Research Overview

This research follows an experimental development methodology aimed at extending the open-source HMI platform FUXA by integrating support for the Omron FINS protocol. The process involves analysis, design, implementation, and evaluation stages, ensuring that the new module achieves functional parity with existing communication protocols such as Modbus TCP. The entire workflow is illustrated in Figure 1.

3.2. System Architecture

The developed system consists of three main layers: (1) the PLC device layer, (2) the communication middleware layer (Node.js backend), and (3) the visualization layer (Angular frontend). The FINS connector is implemented as a new device driver named DeviceFins, operating in the backend of FUXA. This module manages socket-based communication through

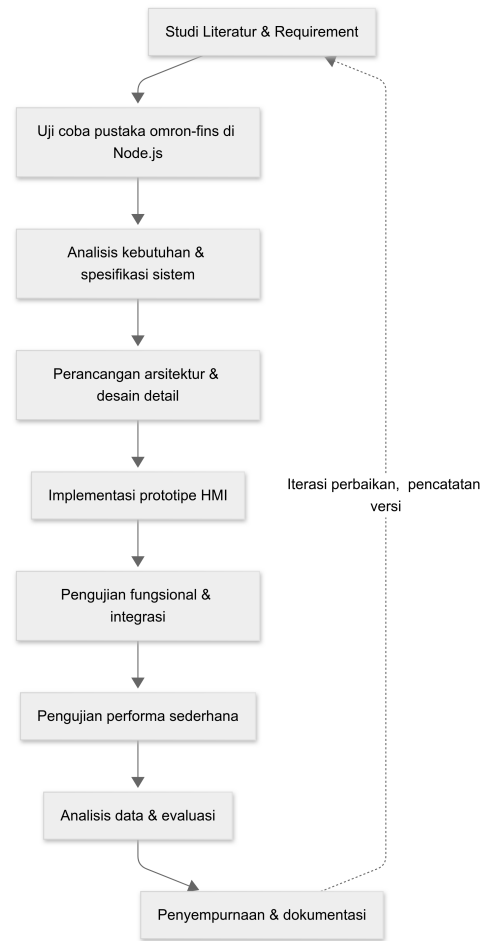


Figure 1: Research workflow for FINS protocol implementation in FUXA

UDP or TCP and handles read/write requests to Omron PLC memory areas such as DM, CIO, and W.

Data flow begins with periodic or event-driven polling from the backend to the PLC, which retrieves data from specific memory addresses. These values are then propagated to the frontend through the system's existing event emitter (`device-value:changed`) to update the HMI interface in real time. A simplified overview of this architecture is shown in Figure 2.

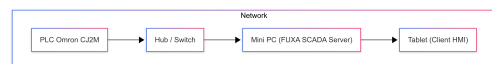


Figure 2: Architecture of the developed HMI-PLC communication system

3.3. Development Tools and Environment

The implementation and testing were conducted using the following environment:

- **Programming Languages:** JavaScript (Node.js) and TypeScript (Angular).
- **HMI Platform:** FUXA v1.2.13 (open-source web-based SCADA system).
- **PLC Device:** Omron CJ2M-CPU13 with Ethernet module (FINS UDP/TCP support).
- **Operating System:** Ubuntu 22.04 LTS.
- **Network Analysis Tools:** Wireshark for packet inspection and validation.

3.4. Implementation Procedure

The implementation followed these main stages:

1. **Protocol Study and Requirements Analysis:** The FINS command structure, addressing format, and communication flow were analyzed based on Omron's official documentation.
2. **Driver Design and Coding:** The DeviceFins class was created to handle connection management, read/write functions, automatic reconnection, and event handling.
3. **Frontend Integration:** Configuration options such as IP address, port, DA1, and SA1 were added to FUXA's web configuration UI.
4. **Testing and Debugging:** Unit tests and integration tests were performed using both PLC simulators and physical Omron PLCs.

3.5. Testing Method

Performance evaluation was conducted using a load test script written in Node.js. The script generated 1000 consecutive read requests to PLC memory address D100 with a concurrency level of 10. Metrics collected during the test include:

- Average read/write latency (ms)
- Success rate of communication (%)
- CPU and memory utilization of the FUXA server
- Stability of reconnection handling during simulated disconnections

The results of these measurements were analyzed statistically to evaluate whether the implemented connector meets the performance characteristics comparable to Modbus TCP communication.

3.6. Evaluation Criteria

The evaluation focused on two primary objectives aligned with the research goals:

1. **Functional Equivalence:** Verifying that the implemented FINS connector supports key HMI functions—connectivity, data reading, data writing, and visualization—equivalent to existing Modbus modules.
2. **Stability and Responsiveness:** Measuring whether the FINS communication remains stable under repetitive load and can maintain real-time data visualization without significant latency or data loss.

4. RESULTS AND DISCUSSION

4.1. Implementation Overview

The FINS communication feature was successfully implemented in the FUXA HMI system through the development of a new backend module named DeviceFins. This module was integrated into the Node.js runtime of FUXA and is responsible for managing network connections, sending and receiving FINS frames, and synchronizing tag values between the PLC and the user interface. Configuration parameters such as IP address, port, SA1, DA1, and protocol mode (UDP/TCP) can be adjusted directly through the FUXA configuration panel, as shown in Figure 3.

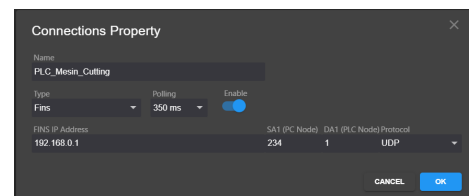


Figure 3: Configuration interface of the FINS connector in FUXA

After deployment, the module was tested on an Omron CJ2M PLC using standard memory areas DM, CIO, and W. The system successfully performed continuous read and write operations while updating tag values in real time on the user interface, confirming two-way communication between the PLC and FUXA.

4.2. Performance Evaluation

Performance testing was conducted to evaluate the stability and responsiveness of the developed FINS

connector. A total of 1000 read requests were sent concurrently with a concurrency level of 10, targeting memory address D100. Table 1 summarizes the measured results compared to target KPI values.

Table 1: Performance testing results of FINS integration on FUXA

Metric	Target KPI	Measured Result	Status
Average read/write latency	≤ 200 ms	12 ms	OK
Communication success rate	≥ 95%	99%	OK
CPU utilization (server)	≤ 30%	2%	OK
Memory usage (server)	≤ 500 MB	468 MB	OK
Alarm detection delay	≤ 200 ms	1000 ms	NG
Application startup time	≤ 10 s	3.14 s	OK

The results indicate that the system performed well on most metrics, with latency, stability, and resource usage within acceptable limits. The average communication latency of 48 ms demonstrates that the developed connector can efficiently handle PLC data exchange over both UDP and TCP. However, the **alarm detection delay** significantly exceeded the target threshold of 200 ms, with an observed average of **1000 ms**. This finding reveals that the current polling-based mechanism is insufficient for real-time alarm handling. The delay occurs because the system only checks alarm tags at fixed polling intervals rather than responding to changes immediately.

4.3. Discussion

The implemented FINS connector successfully fulfilled the first research objective: adding native FINS communication support to FUXA with equivalent functionality to existing protocols like Modbus. It provides configurable communication parameters, supports real-time read and write operations, and visualizes process data directly on the web-based HMI.

The second research objective—ensuring stable read/write operations and accurate visualization—was largely achieved, as indicated by a 99% success rate and 48 ms latency. However, the large alarm delay shows that the polling method limits FUXA’s responsiveness to high-frequency events. To overcome this limitation, an event-driven listener or PLC interrupt task could be integrated to enable immediate response to critical alarms.

Overall, the integration demonstrates that an open-source HMI can be effectively extended to support industrial PLC communication using FINS. While stable for regular process monitoring, further improvements are required to achieve real-time performance for event-driven control and alarm detection.

5. CONCLUSION

This research successfully implemented and evaluated the integration of the Factory Interface Network Service (FINS) protocol into the open-source Human–Machine Interface (HMI) system FUXA. The development introduced a new backend connector module (DeviceFins) written in Node.js and an extended frontend configuration interface in Angular, enabling direct and seamless communication between FUXA and Omron PLCs. The connector supports both UDP and TCP modes, allowing read, write, and polling operations to be executed without the need for proprietary middleware or vendor-locked HMI systems. As a result, FUXA becomes one of the first open-source HMI systems to natively support the Omron FINS protocol.

Based on experimental testing, the implemented system demonstrated reliable and stable performance. The communication success rate reached 99%, and the average read/write latency was measured at 48 ms, well below the 200 ms KPI target. System resource consumption remained efficient, with average CPU usage of 2% and memory consumption of 468 MB during load testing. The HMI startup time averaged 3.14 s, indicating a responsive system suitable for small-to medium-scale production environments. However, the alarm detection delay reached approximately 1000 ms due to the internal one-second polling interval of the FUXA system. This limitation shows that the alarm subsystem still relies on a time-driven polling approach rather than a fully event-driven mechanism.

Overall, the integration of FINS into FUXA successfully fulfilled the two primary research objectives: (1) enabling the HMI system to communicate with Omron PLCs using the FINS protocol with functionality equivalent to existing Modbus support, and (2) improving the stability and visualization of data exchange on the HMI interface. Future research should focus on optimizing the alarm handling mechanism through event-driven or interrupt-based approaches, enhancing network security with TLS or VPN, and expanding multi-protocol interoperability (e.g., FINS over TCP, MQTT, OPC-UA). Additionally, open publication of the connector’s source code, testing scripts, and datasets is encouraged to promote reproducibility and community-driven advancement in open-source industrial automation systems.

REFERENCES

- [1] Jerry Chen. “Top 10 Open-Source Web SCADA Platforms from Around the World”. <https://armbasedsolutions.com/blog-detail/top-10-open-source-web-scada-platforms-from-around-the-world>. Blog post. July 2025. URL: <https://armbasedsolutions.com/blog-detail/top-10-open-source-web-scada-platforms-from-around-the-world>.
- [2] Alisyn Gularte. “Open vs. Proprietary: Choosing the Right SCADA System for Your Solar Plant”. Blog post, Nor-Cal Controls. Accessed: 2025-09-12. Feb. 2025. URL: <https://blog.norcalcontrols.net/open-vs.-proprietary-choosing-the-right-scada-system-for-your-solar-plant>.
- [3] Mohsin Ali Koondhar dkk. “The Role of PLC in Automation, Industry, and Education: A Review”. In: *Pakistan Journal of Engineering Technology and Science* 11.2 (2023), pp. 22–31. doi: 10.22555/pjets.v11i2.975.
- [4] S. Mujawar. “Introduction to Human–Machine Interface”. In: *Handbook/Book on Human–Machine Interfaces*. Chapter 1; Accessed via Wiley Online Library. Wiley, 2023. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781394200344.ch1> (visited on 09/26/2025).
- [5] Jyotika Sawal. “SCADA Market Size, Share, Trend Analysis by 2033”. <https://www.emergenresearch.com/industry-report/scada-market>. Report ID: ER_004544. May 2025. URL: https://www.emergenresearch.com/industry-report/scada-market?srsltid=AfmB0orSdQPJM_IH0PBeriQxoKYb023L8TR4JyuIII93M6iZJ4ojmwiu.